

The Digital Mirror

VOL. 1, NO. 085

2026-03-30

COVER STORY

Rewriting pycparser with the help of an LLM

pycparser is my most widely used open source project (with 20M daily downloads from PyPI [1]).

Eli Bendersky

It's a pure-Python parser for the C programming language, producing ASTs inspired by Python's own . Until very recently, it's been using `PLY`: Python Lex-Yacc for the core parsing.

In this post, I'll describe how I collaborated with an LLM coding agent (Codex) to help me rewrite pycparser to use a hand-written recursive-descent parser and remove the dependency on `PLY`. This has been an interesting experience and the post contains lots of information and is therefore quite long; if you're just interested in the final result, check out the latest code of pycparser - the main branch already has the new implementation.

The issues with the existing parser implementation

While pycparser has been working well overall, there were a number of nagging issues that persisted over years.

Parsing strategy: YACC vs. hand-written recursive descent

I began working on pycparser in 2008, and back then using a YACC-based approach for parsing a whole language like C seemed like a no-brainer to me. Isn't this what everyone does when writing a serious parser? Besides, the K&R2 book famously carries the entire grammar of the C99 language in an appendix - so it seemed like a simple matter of translating that to `PLY`-yacc syntax.

And indeed, it wasn't too hard, though there definitely were some complications in building the ASTs for declarations (C's gnarliest part).

Shortly after completing pycparser, I got more and more interested in compilation and started learning about the different kinds of parsers more seriously. Over time, I grew convinced that recursive descent is the way to go - producing parsers that are easier to understand and maintain (and are often faster!).

It all ties in to the benefits of dependencies in software projects as a function of effort . Using parser generators is a heavy conceptual dependency: it's really nice when you have to churn out many parsers for small languages. But when you have to maintain a single, very complex parser, as part of a large project - the benefits quickly dissipate and you're left with a substantial dependency that you constantly grapple with.

The other issue with dependencies

And then there are the usual problems with dependencies; dependencies get abandoned, and they may also develop security issues. Sometimes, both of these become true.

Many years ago, pycparser forked and started vendoring its own version of `PLY`. This was part of transitioning pycparser to a dual Python 2/3 code base when `PLY` was slower to adapt. I believe this was the right decision, since `PLY` "just worked" and I didn't have to deal with active (and very tedious in the Python ecosystem, where packaging tools are replaced faster than dirty socks) dependency management.

A couple of weeks ago this issue was opened for pycparser. It turns out the some old `PLY` code triggers security checks used by some Linux distributions; while this code was fixed in a later commit of `PLY`, `PLY` itself was apparently abandoned and archived in late 2025. And guess what? That happened in the middle of a large rewrite of the package, so re-vendoring the pre-archiving commit seemed like a risky proposition.

On the issue it was suggested that "hopefully the dependent packages move on to a non-abandoned parser or imple-

ment their own”; I originally laughed this idea off, but then it got me thinking... which is what this post is all about.

Growing complexity of parsing a messy language

The original K&R2 grammar for C99 had - famously - a single shift-reduce conflict having to do with dangling else s belonging to the most recent if statement. And indeed, other than the famous lexer hack used to deal with C’s type name / ID ambiguity , pycparser only had this single shift-reduce conflict.

But things got more complicated. Over the years, features were added that weren’t strictly in the standard but were supported by all the industrial compilers. The more advanced C11 and C23 standards weren’t beholden to the promises of conflict-free YACC parsing (since almost no industrial-strength compilers use YACC at this point), so all caution went out of the window.

The latest (PLY-based) release of pycparser has many reduce-reduce conflicts [2] ; these are a severe maintenance hazard because it means the parsing rules essentially have to be tie-broken by order of appearance in the code. This is very brittle; pycparser has only managed to maintain its stability and quality through its comprehensive test suite. Over time, it became harder and harder to extend, because YACC parsing rules have all kinds of spooky-action-at-a-distance effects. The straw that broke the camel’s back was this PR which again proposed to increase the number of reduce-reduce conflicts [3] .

This - again - prompted me to think “what if I just dump YACC and switch to a hand-written recursive descent parser”, and here we are.

The mental roadblock

None of the challenges described above are new; I’ve been pondering them for many years now, and yet biting the bullet and rewriting the parser didn’t feel like something I’d like to get into. By my private estimates it’d take at least a week of deep heads-down work to port the gritty 2000 lines of YACC grammar rules to a recursive descent parser [4] . Moreover, it wouldn’t be a particularly fun project either - I didn’t feel like I’d learn much new and my interests have shifted away from this project. In short, the Potential well was just too deep.

Why would this even work? Tests

I’ve definitely noticed the improvement in capabilities of LLM coding agents in the past few months, and many reputable people online rave about using them for increasingly larger projects. That said, would an LLM agent really be able to accomplish such a complex project on its own? This isn’t just a toy, it’s thousands of lines of dense parsing code.

What gave me hope is the concept of conformance suites mentioned by Simon Willison . Agents seem to do well when

there’s a very clear and rigid goal function - such as a large, high-coverage conformance test suite.

And pycparser has an very extensive one . Over 2500 lines of test code parsing various C snippets to ASTs with expected results, grown over a decade and a half of real issues and bugs reported by users.

I figured the LLM can either succeed or fail and throw its hands up in despair, but it’s quite unlikely to produce a wrong port that would still pass all the tests. So I set it to run.

The initial port

I fired up Codex in pycparser’s repository, and wrote this prompt just to make sure it understands me and can run the tests:

```
class="highlight"> This repository implements a Python parser producing an AST. It has a very extensive test suite; can you first verify you successfully run all the tests? (with python3 -m unittest discover, from the root dir)
```

Codex figured it out (I gave it the exact command, after all!); my next prompt was the real thing [5] :

```
class="highlight"> pycparser uses the PLY library for its lexer and parser, and its own AST package for the ASTs created by the parser. I want to explore replacing the Parser component (leaving the Lexer alone for now) with a hand written recursive descent parser that will pass all the same tests. Can you try that?
```

Here Codex went to work and churned for over an hour . Having never observed an agent work for nearly this long, I kind of assumed it went off the rails and will fail sooner or later. So I was rather surprised and skeptical when it eventually came back with:

```
class="highlight"> Replaced the PLY parse path with a hand-written recursive-descent parser while keeping the existing lexer and AST helpers intact; CParser now points to the new RD implementation, with the old PLY version preserved as PLYCParser in pycparser/c_parser.py. The RD parser includes a token stream, declarator/type parsing, expression precedence handling, pragmas, and statement parsing to match the previous AST output and coordinate behavior.
```

It took me a while to poke around the code and run it until I was convinced - it had actually done it! It wrote a new recursive descent parser with only ancillary dependencies on PLY, and that parser passed the test suite. After a few more prompts, we’ve removed the ancillary dependencies and made the structure clearer. I hadn’t looked too deeply into code quality at this point, but at least on the functional level - it succeeded. This was very impressive!

A quick note on reviews and branches

A change like the one described above is impossible to code-review as one PR in any meaningful way; so I used a

different strategy. Before embarking on this path, I created a new branch and once Codex finished the initial rewrite, I committed this change, knowing that I will review it in detail, piece-by-piece later on.

Even though coding agents have their own notion of history and can “revert” certain changes, I felt much safer relying on Git. In the worst case if all of this goes south, I can nuke the branch and it’s as if nothing ever happened. I was determined to only merge this branch onto main once I was fully satisfied with the code. In what follows, I had to git reset several times when I didn’t like the direction in which Codex was going. In hindsight, doing this work in a branch was absolutely the right choice.

The long tail of goofs

Once I’ve sufficiently convinced myself that the new parser is actually working, I used Codex to similarly rewrite the lexer and get rid of the PLY dependency entirely, deleting it from the repository. Then, I started looking more deeply into code quality - reading the code created by Codex and trying to wrap my head around it.

And - oh my - this was quite the journey. Much has been written about the code produced by agents, and much of it seems to be true. Maybe it’s a setting I’m missing (I’m not using my own custom AGENTS.md yet, for instance), but Codex seems to be that eager programmer that wants to get from A to B whatever the cost. Readability, minimalism and code clarity are very much secondary goals.

Using raise...except for control flow? Yep. Abusing Python’s weak typing (like having None, false and other values all mean different things for a given variable)? For sure. Spreading the logic of a complex function all over the place instead of putting all the key parts in a single switch statement? You bet.

Moreover, the agent is hilariously lazy. More than once I had to convince it to do something it initially said is impossible, and even insisted again in follow-up messages. The anthropomorphization here is mildly concerning, to be honest. I could never imagine I would be writing something like the following to a computer, and yet - here we are: “Remember how we moved X to Y before? You can do it again for Z, definitely. Just try”.

My process was to see how I can instruct Codex to fix things, and intervene myself (by rewriting code) as little as possible. I’ve mostly succeeded in this, and did maybe 20% of the work myself.

My branch grew dozens of commits, falling into roughly these categories:

The code in X is too complex; why can’t we do Y instead?

The use of X is needlessly convoluted; change Y to Z, and T to V in all instances.

The code in X is unclear; please add a detailed comment - with examples - to explain what it does.

Interestingly, after doing (3), the agent was often more effective in giving the code a “fresh look” and succeeding in either (1) or (2).

The end result

Eventually, after many hours spent in this process, I was reasonably pleased with the code. It’s far from perfect, of course, but taking the essential complexities into account, it’s something I could see myself maintaining (with or without the help of an agent). I’m sure I’ll find more ways to improve it in the future, but I have a reasonable degree of confidence that this will be doable.

It passes all the tests, so I’ve been able to release a new version (3.00) without major issues so far. The only issue I’ve discovered is that some of CFFI’s tests are overly precise about the phrasing of errors reported by pycparser; this was an easy fix.

The new parser is also faster, by about 30% based on my benchmarks! This is typical of recursive descent when compared with YACC-generated parsers, in my experience. After reviewing the initial rewrite of the lexer, I’ve spent a while instructing Codex on how to make it faster, and it worked reasonably well.

Followup - static typing

While working on this, it became quite obvious that static typing would make the process easier. LLM coding agents really benefit from closed loops with strict guardrails (e.g. a test suite to pass), and type-annotations act as such. For example, had pycparser already been type annotated, Codex would probably not have overloaded values to multiple types (like None vs. False vs. others).

In a followup, I asked Codex to type-annotate pycparser (running checks using ty), and this was also a back-and-forth because the process exposed some issues that needed to be refactored. Time will tell, but hopefully it will make further changes in the project simpler for the agent.

Based on this experience, I’d bet that coding agents will be somewhat more effective in strongly typed languages like Go, TypeScript and especially Rust.

Overall, this project has been a really good experience, and I’m impressed with what modern LLM coding agents can do! While there’s no reason to expect that progress in this domain will stop, even if it does - these are already very useful tools that can significantly improve programmer productivity.

Could I have done this myself, without an agent’s help? Sure. But it would have taken me much longer, assuming that I could even muster the will and concentration to engage in this project. I estimate it would take me at least a week of

full-time work (so 30-40 hours) spread over who knows how long to accomplish. With Codex, I put in an order of magnitude less work into this (around 4-5 hours, I'd estimate) and I'm happy with the result.

anyway - if it succeeds in that, the lexer should be easy. That assumption turned out to be correct.

It was also fun . At least in one sense, my professional life can be described as the pursuit of focus, deep work and flow . It's not easy for me to get into this state, but when I do I'm highly productive and find it very enjoyable. Agents really help me here. When I know I need to write some code and it's hard to get started, asking an agent to write a prototype is a great catalyst for my motivation. Hence the meme at the beginning of the post.

Does code quality even matter?

One can't avoid a nagging question - does the quality of the code produced by agents even matter? Clearly, the agents themselves can understand it (if not today's agent, then at least next year's). Why worry about future maintainability if the agent can maintain it? In other words, does it make sense to just go full vibe-coding?

This is a fair question, and one I don't have an answer to. Right now, for projects I maintain and stand behind , it seems obvious to me that the code should be fully understandable and accepted by me, and the agent is just a tool helping me get to that state more efficiently. It's hard to say what the future holds here; it's going to be interesting, for sure.

[1] pycparser has a fair number of direct dependents , but the majority of downloads comes through CFFI , which itself is a major building block for much of the Python ecosystem.

[2] The table-building report says 177, but that's certainly an over-dramatization because it's common for a single conflict to manifest in several ways.

[3] It didn't help the PR's case that it was almost certainly vibe coded.

[4]

class="first">There was also the lexer to consider, but this seemed like a much simpler job. My impression is that in the early days of computing, lex gained prominence because of strong regexp support which wasn't very common yet. These days, with excellent regexp libraries existing for pretty much every language, the added value of lex over a custom regexp-based lexer isn't very high.

class="last">That said, it wouldn't make much sense to embark on a journey to rewrite just the lexer; the dependency on PLY would still remain, and besides, PLY's lexer and parser are designed to work well together. So it wouldn't help me much without tackling the parser beast.

[5] I've decided to ask it to port the parser first, leaving the lexer alone. This was to split the work into reasonable chunks. Besides, I figured that the parser is the hard job

{

FRONT PAGE

Send Emails In A NativeScript App Via The Mailgun API

Nic Raboy · Nic Raboy (polyglot developer)

I get this question a lot, not necessarily for NativeScript , but for other mobile frameworks. The question is, how do you send emails from within your Android and iOS application without launching one of the already installed mail apps? In other words how do you send emails via a custom form within the application or in the background?

A solution to this question would be to use a backend web server that can send mail and create an API endpoint to it that can be accessed from your mobile app. However, this solution takes work and other resources. Instead you can make use of a transactional email service.

There are many of these email services available, but for this example we're going to look at using the Mailgun service in a NativeScript application.

The post Send Emails In A NativeScript App Via The Mailgun API appeared first on The Polyglot Developer .

}

{

FEATURES

Book Notes: Full Frontal Calculus by Seth Braver — Chapter 1 Review

Ruslan Spivak

“Where there is life, there is change; where there is change, there is calculus.” — Seth Braver

I recently went back to studying math to rebuild my foundations for AI and machine learning. I didn't expect to enjoy a calculus book this much. Shocking, I know. But that's exactly what happened with Full Frontal Calculus.

Can calculus feel intuitive? Even fun? From the first few pages? For me, the answer was yes. (Okay, from page 8 to be exact.)

Why This Book Clicked for Me

As part of my self-study, I'm reviewing select chapters from the books I work through. This post covers Chapter 1 of Full Frontal Calculus by Seth Braver.

Before I stumbled on Full Frontal Calculus, I tried a few limit-based calculus books and textbooks, but none of them spoke to me. Luckily, there's no shortage of calculus material these days, so it's easy to shop around and try different sources.

Braver's book grabbed me right away. The early focus on infinitesimals, the tight writing, and the emphasis on intuition won me over. I even caught myself smiling more than once. Rare for a math book.

Chapter 1 Highlights

Chapter 1 starts with infinitesimals: “an infinitely small number, smaller than any positive real number, yet greater than zero.” One early example shows how a circle, imagined as a polygon with infinitely many infinitesimal sides, leads to the familiar area formula πr^2 . If your geometry or trig is rusty, don't worry - it still makes sense. Braver then uses the same idea to show how curves appear straight on a small enough (infinitesimal) scale, which is the heart of differential calculus.

Things really clicked for me in the section titled A Gift From Leibniz: d-Notation. Braver's explanation of dy/dx shows how it captures infinitesimal change in a way that just makes sense. It helped me understand why derivatives represent slopes and rates in a way I could explain to a 10-year-old. Working through the derivative of x^2 from first principles was also deeply satisfying.

Practically speaking, Chapter 1 covers:

what infinitesimals are

how they help us define rates of change

the geometric meaning of derivatives

the elegant dy/dx notation from Leibniz

why we ignore higher-order infinitesimals like $(dx)^2$ or $du \cdot dv$

and a first-principles derivation of the derivative of x^2

The chapter ends with two powerful tools: the power rule and linearity properties. These let you compute derivatives of polynomials using just basic mental math.

The writing is sharp and often funny, in a math kind of way. There's even a cameo by the Sumerian beer goddess Ninkasi, who helps explain rate of change and derivatives using a vat of beer. It sounds quirky, but it works.

The book's style, clarity, and focus on intuition made me want to keep going. That's not something I've felt with many math books.

Final Thoughts and Tips

If you're following along or just curious about studying calculus again, I recommend giving Chapter 1 a shot. It's not always light reading, and the exercises are essential, but it might click for you like it did for me. Chapter 1 is available for free on the author's site, so you can explore it before deciding whether to dive in.

If you do decide to dive into the book, here are a few tips to get the most out of it:

If you're rusty on pre-calculus (I was), make sure you've got slope, rate of change, the point-slope formula, and the slope-intercept form down cold before the Rates of Change section on page 10. For that, Seth Braver's other book Precalculus Made Difficult has excellent material on those topics. You can probably get through it in a day.

Read slowly, with a pen or pencil in hand. Write in the margins (get a paperback copy). It might feel painfully slow at times (pun intended), but it's a recipe for deeper understanding.

The book includes answers to many exercises and is great for self-study. But the solutions are compact, so I recommend using Grok or ChatGPT to expand on them and deepen your understanding.

Once you've finished the chapter and exercises, check out the author's YouTube videos that go along with the book. They're criminally underrated and oddly hard to find. You might enjoy them as much as I do.

For topics that are hard to retain, try spaced repetition with active recall. Anki works great for that, or use whatever tool you prefer.

Chapter 1 sealed the deal. This is the calculus book I'm sticking with. Looking forward to seeing how Braver develops the ideas from here.

More to come. Stay tuned.

For that, Seth Braver's other book Precalculus Made Difficult has excellent material on those topics.

Ruslan Spivak

}

{

How to Split Ranges in C++23

Bartłomiej Filipiek (C++ Stories)

In this blog post, we'll continue looking at ranges and this time explore ways to split them into sub-ranges. So we'll take a look at `views::split`, `views::chunk`, and `views::chunk_by`.

We'll walk through two examples for each adaptor: one simple and one slightly more advanced, to highlight their practical uses.

Let's go.

`id="splitting-ranges-with-viewssplit-c20"> Splitting Ranges with views::split, C++20`

If you want to split a range using some "delimiter," then `views::split` (or `ranges::split_view`) will do the job.

Both of the ranges `V` and `Pattern` have to be `forward_range`.

Notice that you need to use a pattern; it cannot be a condition.

`id="example-1-splitting-a-sentence-into-words"> Example 1: Splitting a Sentence into Words`

```
class="highlight"> #include #include #include
int main () { using namespace std :: string_view_literals ;
constexpr auto text = "C++ is powerful and elegant" sv ;
for ( auto part : std :: views :: split ( text , ' ' ) ) std :: print ( "{ } " , std :: string_view ( part ) ); }
```

Play @Compiler Explorer

The output:

'C++' 'is' 'powerful' 'and' 'elegant'

A classic example that splits a sentence into words. We convert each subrange to `std::string_view` for easy printing.

We can also extend this and use not just one character:

```
class="highlight"> constexpr auto text = "C++breakis-
breakpowerfulbreakandbreakelegant" sv ;
for ( auto part : std :: views :: split ( text , "break" sv ) ) std ::
print ( "{ } " , std :: string_view ( part ) );
```

`id="example-2-splitting-not-just-text"> Example 2: Splitting Not Just Text`

`split` can be applied to any forward range, so not just text:

```
class="highlight"> #include #include #include #include
int main () { using Point = std :: pair ;
```

```
std :: vector path = { { 0 , 0 } , { 1 , 1 } , { - 1 , - 1 } , /\ marker: { - 1 ,
- 1 } { 2 , 2 } , { 3 , 3 } , { - 1 , - 1 } , { 4 , 4 } , { 5 , 5 } } ;
```

```
for ( auto segment : std :: views :: split ( path , Point { - 1 , -
1 } ) ) std :: print ( "Segment: { } \n " , segment ); }
```

See @Compiler Explorer

And the output:

Segment: [(0, 0), (1, 1)] Segment: [(2, 2), (3, 3)] Segment: [(4, 4), (5, 5)]

In the example, we run through the point list and break at markers that are `(-1, -1)`. This code also leveraged `std::format` support for ranges (C++23), so we don't have to print individual points.

`id="what-about-lazy_split"> What About lazy_split?`

You might also encounter `views::lazy_split`, introduced in C++20.

This view is specialized for input-only ranges, such as reading from streams or generators. It avoids buffering or requiring multiple passes, but its subranges aren't contiguous and don't expose `.data()` or `.size()`.

Unless you're processing streams in a single pass, prefer `views::split` for simplicity.

Here's a question from Stack Overflow that I recently asked to clarify this view: `c++ - What is the use of std::ranges::views::lazy_split when we have std::ranges::views::split?` - Stack Overflow

`id="grouping-with-viewschunk-c23"> Grouping with views::chunk, C++23`

When you need to process data in fixed-size batches, `chunk` is the perfect tool.

`id="example-1-fixed-size-batches"> Example 1: Fixed-Size Batches`

```
class="highlight"> #include #include #include
int main () { std :: vector data { 1 , 2 , 3 , 4 , 5 , 6 , 7 , 8 } ;
for ( auto chunk : data | std :: views :: chunk ( 3 ) ) std :: print
( "{ } \n " , chunk ); }
```

See @Compiler Explorer

And the output:

[1 2 3] [4 5 6] [7 8]

`views::chunk` splits the sequence into groups of three elements. If the number of elements isn't divisible by 3, the last chunk will contain fewer elements.

`id="example-2-processing-network-packets-in-chunks"> Example 2: Processing Network Packets in Chunks`

}

{

5 to 18: Why Your Count Might Be Off by One

Ruslan Spivak

Quick question:

How many numbers are there from 5 to 18, including both ends?

Your first instinct might be to subtract:

$$18 - 5 = 13$$

Feels right.

But it's wrong.

It's a small thing, and kind of basic, but this mistake got me more times than I'd like to admit. Eventually I learned how to count ranges properly. :)

id="basic-counting">Basic counting

Let's start with something simpler.

How many numbers are in this list?

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13

You'd probably say 13 without counting. And you'd be absolutely right.

That kind of range is easy. Our brain sees the pattern and knows what's going on. We've been counting this way since we were little kids. But that instinct quietly fails in cases like 5 to 18.

So, how many numbers are there between 5 and 18 inclusive?

id="lets-count-manually">Let's count manually

Here's the full list from 5 to 18:

5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18

That's 14 numbers, not 13. So what went wrong with our subtraction?

id="try-a-quick-trick">Try a quick trick

Let's take the above list from 5 to 18 and turn it into a list that we know how to count by subtracting 4 from every number (I first saw this approach in David Patrick's book Introduction to Counting & Probability. A great resource if you enjoy these kinds of problems.):

Much easier to count! It has 14 items. Since we only shifted the numbers to start at 1 (without changing the count), the original list has 14 numbers too. Nice.

id="the-formula">The formula

If you want to count how many numbers are in a list from a to b , inclusive, here's the rule (given that both a and b are positive and $b \geq a$):

That +1 is the key.

So for our original example:

$$18 - 5 + 1 = 14$$

id="for-the-curious-formula-derivation">For the curious: formula derivation

Using the same trick, we subtract $a - 1$ from each number in the range from a to b . This transforms it into a list we can count easily, starting from 1:

Our new list has $b - a + 1$ numbers, so the original list has the same count.

The +1 rule is simple, but surprisingly easy to overlook. Here's where it often sneaks in.

id="where-this-trips-people-up">Where this trips people up

This tiny +1 mistake shows up everywhere:

Days between two calendar dates (inclusive)

Characters in a string or line

Steps in a workout plan

Floors between 3 and 7 (did you forget to count the 3rd floor?)

Loop boundaries in code ($\leq$ vs $<$)

Miss it, and you might:

Pack too few t-shirts for your vacation

Underestimate your timeline

Overrun an array and ship a bug

It's such a common mistake, it even has a name: the off-by-one error . Software engineers run into it constantly, but it affects everyone.

Once you understand it, you'll start spotting it everywhere. In code, in calendars, in life.

P. S. If you feel extra adventurous, try the following exercises:

How many numbers are in the range from 42 to 58, inclusive?

Derive the formula $b - a + 1$ from scratch, no peeking

But that instinct quietly fails in cases like 5 to 18.

Ruslan Spivak

}

{

Send More Investor Updates

Zach Holman

I've made like a hundred angel investments over the years, and I get the sneaking suspicion that the most successful startups in my portfolios are the ones that talk to their investors.

Like, that's it. That's the nugget for this post.

id="holy-shit-no-one-can-help-you-if-they-dont-know-you-need-help">Holy shit no one can help you if they don't know you need help

Like most good advice, this is kind of obvious, but writing about it just serves as a reminder and/or a kick in the pants. To wit: it's hard to give help if you don't know what help is needed.

Let's back up a second. First: venture-backed companies are relatively rare, in the grand scheme of things, and that's fine and dandy. But if you are one of the companies where it makes sense to take outside investment, just cashing the check and moving on is pretty small-minded. The whole point of selling part of your company is to bring on people with a network and experience different from your own. If you're not doing that... take out a loan or rob a bank.

Lead investors tend to be much more involved in a company- they might have prescheduled check-in periods and have a deeper understanding of what's going on. But for everyone else, particularly angel investors, the monthly investor update is the only real connection they have to your company, past simply existing on your cap table. Use and leverage them!

id="less-is-more">Less is more

I swear, if I searched in my inbox for "sorry it's been awhile since we sent an investor update!" the results would be... high. And I get it! It's kind of stressful, particularly if you have stressful things in your present or your future. And usually it just keeps compounding until the dam bursts and once a year there's a "OHMYGOD HERE'S A WAR AND PEACE -SIZED TREATISE ON THE COMPANY BECAUSE IT'S BEEN SO LONG ALSO PS: WE'VE PIVOTED TO SELLING SMALL TREES FOR CATS TO PLAY ON".

So your easy answer: just send a quick paragraph! Nothing needs to be rocket science. I've gotten a number of updates that broadly consisted of:

Hi! This is what we did this last month:

[some cool hip things]

Here's what we need help with this upcoming month:

Nothing! We're doing good, just heads-down on the product right now.

I love these! This tells me 1) what you did, 2) what you need help on (nothing!), 3) that you're still alive, which you'd be shocked at how often that's a real question. I've had a couple investments now where the only communications were "thanks for your investment!" and "we're shutting down the company!" In hindsight: absolutely not shocked the company died.

id="better-companies-communicate">Better companies communicate

Look: I don't traffic in shit like "science" or "math", much less call it "maths" like those weird loveable Brits keep trying to push. I'm an angel investor, baby, so that means all I need are vibes , mannnnn. But I keep getting the feels that founders who are on top of their investor updates are... better.

Almost certainly the best investor updates with the most regular cadence over years was from Sid Sijbrandij , for GitLab's investor updates. There was a very credible rumor on the street that NIST sets their time servers based on Sid's email cadence. They were to the point, plaintext (don't get me started on all the fancy investor updates founders are trying to get by with these days), with clear call-to-actions and jump-off links if I wanted to dig deeper into that particular topic or internal discussion.

I'm not going to say if you take Sid's same approach you're also going to also have a \$15B IPO like GitLab did... but I'm also not not saying it.

So give your investors a shout. Even if I invested in your company and it's been awhile but you don't want to send an investor update after reading this because you're worried it's now uncool to send one out right this minute because other companies are prolly going to do it and you want to be a unique cool snowflake? I definitely wrote this for you. Just send it! You're going to forget it if you push it out a few days anyway. Promise.

}

{

Let's Build A Simple Interpreter. Part 7: Abstract Syntax Trees

Ruslan Spivak

As I promised you last time, today I will talk about one of the central data structures that we'll use throughout the rest of the series, so buckle up and let's go.

Up until now, we had our interpreter and parser code mixed together and the interpreter would evaluate an expression as soon as the parser recognized a certain language construct like addition, subtraction, multiplication, or division. Such interpreters are called syntax-directed interpreters. They usually make a single pass over the input and are suitable for basic language applications. In order to analyze more complex Pascal programming language constructs, we need to build an intermediate representation (IR). Our parser will be responsible for building an IR and our interpreter will use it to interpret the input represented as the IR.

It turns out that a tree is a very suitable data structure for an IR.

Let's quickly talk about tree terminology.

A tree is a data structure that consists of one or more nodes organized into a hierarchy.

The tree has one root, which is the top node.

All nodes except the root have a unique parent.

The node labeled `*` in the picture below is a parent. Nodes labeled 2 and 7 are its children; children are ordered from left to right.

A node with no children is called a leaf node.

A node that has one or more children and that is not the root is called an interior node.

The children can also be complete subtrees. In the picture below the left child (labeled `*`) of the `+` node is a complete subtree with its own children.

In computer science we draw trees upside down starting with the root node at the top and branches growing downward.

Here is a tree for the expression $2 * 7 + 3$ with explanations:

The IR we'll use throughout the series is called an abstract-syntax tree (AST). But before we dig deeper into ASTs let's talk about parse trees briefly. Though we're not going to use parse trees for our interpreter and compiler, they can help you understand how your parser interpreted the input by visualizing the execution trace of the parser. We'll also compare them with ASTs to see why ASTs are better suited for intermediate representation than parse trees.

So, what is a parse tree? A parse-tree (sometimes called a concrete syntax tree) is a tree that represents the syntactic structure of a language construct according to our grammar definition. It basically shows how your parser recognized the language construct or, in other words, it shows how the start symbol of your grammar derives a certain string in the programming language.

The call stack of the parser implicitly represents a parse tree and it's automatically built in memory by your parser as it is trying to recognize a certain language construct.

Let's take a look at a parse tree for the expression $2 * 7 + 3$:

In the picture above you can see that:

The parse tree records a sequence of rules the parser applies to recognize the input.

The root of the parse tree is labeled with the grammar start symbol.

Each interior node represents a non-terminal, that is it represents a grammar rule application, like `expr`, `term`, or `factor` in our case.

Each leaf node represents a token.

As I've already mentioned, we're not going to manually construct parser trees and use them for our interpreter but parse trees can help you understand how the parser interpreted the input by visualizing the parser call sequence.

You can see how parse trees look like for different arithmetic expressions by trying out a small utility called `genptdot.py` that I quickly wrote to help you visualize them. To use the utility you first need to install `Graphviz` package and after you've run the following command, you can open the generated image file `parsetree.png` and see a parse tree for the expression you passed as a command line argument:

Here is the generated image `parsetree.png` for the expression $14 + 2 * 3 - 6 / 2$:

Play with the utility a bit by passing it different arithmetic expressions and see what a parse tree looks like for a particular expression.

Now, let's talk about abstract-syntax trees (AST). This is the intermediate representation (IR) that we'll heavily use throughout the rest of the series. It is one of the central data structures for our interpreter and future compiler projects.

Let's start our discussion by taking a look at both the AST and the parse tree for the expression $2 * 7 + 3$:

As you can see from the picture above, the AST captures the essence of the input while being smaller.

Here are the main differences between ASTs and Parse trees:

}

{

22 Common Filesystem Tasks in C++20

Bartłomiej Filipek (C++ Stories)

Working with the filesystem can be a daunting task, but it doesn't have to be. In this post, I'll walk you through some of the most common filesystem operations using the powerful features introduced in C++17, as well as some new enhancements in C++20/23. Whether you're creating directories, copying files, or managing permissions, these examples will help you understand and efficiently utilize the `std::filesystem` library.

Let's start.

id="directory-operations"> Directory Operations

id="1-creating-directories">

1. Creating Directories

Creating directories is a basic yet essential operation. The `std::filesystem` library makes this straightforward with the `create_directory` function.

```
class="highlight"> #include #include
```

```
int main () { std :: filesystem :: path dir =  
"example_directory"; if ( std :: filesystem :: create_directory  
( dir )) std :: cout create_directory ( const std :: filesys-  
tem :: path & p , std :: error_code & ec ) noexcept ;
```

For example:

```
class="highlight"> #include #include
```

```
int main () { std :: filesystem :: path dir = "abc/cde" ; /  
 \ recursive? std :: error_code ec {} ; if ( std :: filesystem ::  
create_directory ( dir , ec )) std :: cout Failed to create direc-  
tory: No such file or directory ...
```

See more @Cppreference

id="2-creating-nested-directories">

2. Creating Nested Directories

When you need to create multiple nested directories, use the `create_directories` function.

This function creates directories recursively.

```
class="highlight"> #include #include #include
```

```
int main () { std :: filesystem :: path nested = "a/b/c" ; try { if  
( std :: filesystem :: create_directories ( nested )) std :: cout  
3. Removing a Directory
```

So far, our example directories were left in a vacuum (and I hope Compiler Explorer can safely remove them :), but we can also do it manually from code: We just need the `remove` function for this task.

```
class="highlight"> #include #include #include
```

```
void ls () { ... }
```

```
int main () { std :: filesystem :: path dir = "test" ; try { if ( std ::  
filesystem :: create_directory ( dir )) std :: cout Directory  
created successfully "./compilation-result.json" ".output.s"  
"./example.cpp" ".test" Directory removed "./compilation-  
result.json" ".output.s" ".example.cpp"
```

id="4-removing-all-contents-of-a-directory">

4. Removing All Contents of a Directory

To remove a directory along with all its contents, use the `remove_all` function.

```
class="highlight"> #include #include #include #include
```

```
void ls () { for ( const auto & entry : std :: filesystem ::  
recursive_directory_iterator ( "." )) std :: cout Directories  
created successfully "./compilation-result.json" ".output.s"  
"./example.cpp" ".test" ".test/x" ".test/x/y" ".test/a" "./  
test/a/b" removed 5 items "./compilation-result.json" "./  
output.s" ".example.cpp"
```

Notice that we had `test/x/y`, `test/x`, `test/a/b`, `test/a` and `test` directories to remove in this case.

id="5-listing-directory-contents">

5. Listing Directory Contents

In my previous examples, I've used the magical `ls()` function, but the implementation is quite simple. It uses `directory_iterator` to iterate through the directory:

id="6-listing-directory-contents-recursively">

6. Listing Directory Contents Recursively

We can use a recursive version of the `directory_iterator` to iterate through a whole subtree:

```
class="highlight"> #include #include
```

```
void ls () { for ( const auto & entry : std :: filesystem ::  
recursive_directory_iterator ( "." )) std :: cout ".compilation-  
result.json" ".output.s" ".x" ".x/y" ".x/y/z" ".example.cpp"  
"./a" ".a/b" ".a/b/c"
```

If we need a nicer output, we can try implementing our own recursive version:

```
class="highlight"> #include #include
```

```
void ls ( const std :: filesystem :: path & p , unsigned  
tabs = 0 ) { for ( const auto & entry : std :: filesystem ::  
directory_iterator ( p )) { if ( tabs > 0 ) std :: cout "compilation-  
result" "output" "x" - "y" - "z" "example" "a" - "b" - "c"
```

id="7-creating-and-using-a-temporary-directory">

7. Creating and Using a Temporary Directory

Thus far, I have used the current directory to create new files and test dirs, but the `std::filesystem` library provides facilities to create and manage temporary directories in a better way. It all comes down to the `temp_directory_path()`

}

{

Let's Build A Web Server. Part 1.

Ruslan Spivak

Out for a walk one day, a woman came across a construction site and saw three men working.

She asked the first man, "What are you doing?"

Annoyed by the question, the first man barked, "Can't you see that I'm laying bricks?" Not satisfied with the answer, she asked the second man what he was doing.

The second man answered, "I'm building a brick wall." Then, turning his attention to the first man, he said, "Hey, you just passed the end of the wall. You need to take off that last brick." Again not satisfied with the answer, she asked the third man what he was doing. And the man said to her while looking up in the sky, "I am building the biggest cathedral this world has ever known." While he was standing there and looking up in the sky the other two men started arguing about the errant brick. The man turned to the first two men and said, "Hey guys, don't worry about that brick. It's an inside wall, it will get plastered over and no one will ever see that brick. Just move on to another layer."

The moral of the story is that when you know the whole system and understand how different pieces fit together (bricks, walls, cathedral), you can identify and fix problems faster (errant brick).

What does it have to do with creating your own Web server from scratch?

I believe to become a better developer you MUST get a better understanding of the underlying software systems you use on a daily basis and that includes programming languages, compilers and interpreters, databases and operating systems, web servers and web frameworks. And, to get a better and deeper understanding of those systems you MUST rebuild them from scratch, brick by brick, wall by wall.

Confucius put it this way:

"I hear and I forget."

"I see and I remember."

"I do and I understand."

I hope at this point you're convinced that it's a good idea to start re-building different software systems to learn how they work.

In this three-part series I will show you how to build your own basic Web server. Let's get started.

First things first, what is a Web server?

In a nutshell it's a networking server that sits on a physical server (oops, a server on a server) and waits for a client to send a request. When it receives a request, it generates

a response and sends it back to the client. The communication between a client and a server happens using HTTP protocol. A client can be your browser or any other software that speaks HTTP.

What would a very simple implementation of a Web server look like? Here is my take on it. The example is in Python (tested on Python3.7+) but even if you don't know Python (it's a very easy language to pick up, try it!) you still should be able to understand concepts from the code and explanations below:

HOST, PORT = " , 8888

```
listen_socket = socket . socket ( socket . AF_INET , socket .
SOCK_STREAM ) listen_socket . setsockopt ( socket .
SOL_SOCKET , socket . SO_REUSEADDR , 1 ) listen_socket .
bind ( ( HOST , PORT ) ) listen_socket . listen ( 1 ) print
( f 'Serving HTTP on port { PORT } ...' ) while True :
client_connection , client_address = listen_socket . accept
() request_data = client_connection . recv ( 1024 ) print
( request_data . decode ( 'utf-8' ) )
```

http_response = b "" "\ HTTP/1.1 200 OK

```
Hello, World! "" client_connection . sendall
( http_response ) client_connection . close ()
```

Save the above code as webserver1.py or download it directly from GitHub and run it on the command line like this

Now type in the following URL in your Web browser's address bar `http://localhost:8888/hello` , hit Enter, and see magic in action. You should see "Hello, World!" displayed in your browser like this:

Just do it, seriously. I will wait for you while you're testing it.

Done? Great. Now let's discuss how it all actually works.

First let's start with the Web address you've entered. It's called an URL and here is its basic structure:

This is how you tell your browser the address of the Web server it needs to find and connect to and the page (path) on the server to fetch for you. Before your browser can send a HTTP request though, it first needs to establish a TCP connection with the Web server. Then it sends an HTTP request over the TCP connection to the server and waits for the server to send an HTTP response back. And when your browser receives the response it displays it, in this case it displays "Hello, World!"

Let's explore in more detail how the client and the server establish a TCP connection before sending HTTP requests and responses. To do that they both use so-called sockets . Instead of using a browser directly you are going to simulate your browser manually by using telnet on the command line.

On the same computer you're running the Web server fire up a telnet session on the command line specifying a host

}

standard-article(title: [Getting Started With Capacitor Using React], author: [Roman Akhromieiev], source-name: [Nic Raboy (polyglot developer)], images: (), paragraphs: ([In February 2018 the Ionic team announced Capacitor . Capacitor allows you to make your frontend builds run everywhere, for example, iOS, Android, Electron and in the browser as a progressive web application (PWA).], [Also, Capacitor provides a high-quality API to work with native functions within your apps. For example, you can get access to the camera and it means you still get this access to every platform.], [In the official Capacitor documentation you can find a well-described tutorial on how to use Capacitor with Ionic, but in our tutorial, we will check how you can add Capacitor to your existing React app. We will create a simple React app with Capacitor and run it on iOS and in the browser as PWA.], [The post Getting Started With Capacitor Using React appeared first on The Polyglot Developer .],), insert-map: (:), word-count: 138, edited-for-length: false, debug-mode: false,)

standard-article(title: [Validating Data Structures And Variables In Golang], author: [Nic Raboy], source-name: [Nic Raboy (polyglot developer)], images: (), paragraphs: ([When working with Go or any programming language for that matter, there is almost always a need to validate the data that the user provides before you start working with it or storing it in a database. A sloppy way to validate data would be to use a series of if/else conditions, switch statements, and a bunch of regular expressions, but there are better ways to get the job done without having a disaster of a codebase in terms of maintainability.], [We're going to see how to use the validator.v9 package in Golang to validate native Go data structures, their fields, and any variables that don't quite fit in.], [The post Validating Data Structures And Variables In Golang appeared first on The Polyglot Developer .],), insert-map: (:), word-count: 125, edited-for-length: false, debug-mode: false,)

standard-article(title: [Announcing The Polyglot Developer Courses Portal], author: [Nic Raboy], source-name: [Nic Raboy (polyglot developer)], images: (), paragraphs: ([I'm pleased to announce that The Polyglot Developer has its own courses portal , a replacement to Udemy and Gumroad!], [The new portal, powered by Teachable , is a dedicated area for development courses relating to the material typically found on The Polyglot Developer blog. The portal will contain a variety of courses, some of which are free, and some of which are paid, but all of which are offering a premium learning experience.], [I wanted to take a moment to explain what you'll be able to find in the courses portal

standard-article(title: [Using Socket.io to Create a Multiplayer Game with Angular and Node.js], author: [Nic Raboy], source-name: [Nic Raboy (polyglot developer)], images: (), paragraphs: ([When it comes to client and server communication, there are quite a few approaches towards solving the problem. You could create a RESTful API or GraphQL API on your server and consume it on-demand with your client, or you can go the socket approach and interact in real-time through events. There isn't a wrong way to do things, but some ways are better than others given the task at hand.], [Let's take for example gaming and online multiplayer games. While certain aspects of the game would make sense to use REST or GraphQL, not everything would feel responsive enough. Instead it makes sense to use sockets.], [In this tutorial we're going to explore Socket.io for our client and server communication. We're going to create a Socket.io server with Node.js and that server will communicate with each of our games running as Angular applications.], [The post Using Socket.io to Create a Multiplayer Game with Angular and Node.js appeared first on The Polyglot Developer .],), insert-map: (:), word-count: 163, edited-for-length: false, debug-mode: false,)

standard-article(title: [Add Pagination to Your Eleventy Static Generated Website in Minutes], author: [Nic Raboy], source-name: [Nic Raboy (polyglot developer)], images: (), paragraphs: ([A few months ago you might remember a tutorial I put out regarding remote caching in Eleventy. In this tutorial titled, Download and Cache YouTube Data in an Eleventy Website with Simple JavaScript , I demonstrated how I was automatically gathering YouTube videos from a playlist on my Poké Trainer Nic channel and publishing them to my Eleventy website.], [At the time, this was the best thing since sliced bread for that particular website. Fast forward to now and we've got a problem with too many videos loading all at once with a ten hour scroll to reach the bottom. The scroll time is an over exaggeration, but you get the idea that too much content on a single page can become a problem.], [This lead me to pagination and showing only a small subset of videos per page.], [We're going to see how to very quickly add pagination to an Eleventy website, something that can be accomplished with a few lines of HTML and a few minutes of your time.], [The post Add Pagination to Your Eleventy Static Generated Website in Minutes appeared first on The Polyglot Developer .],), insert-map: (:), word-count: 190, edited-for-length: false, debug-mode: false,)

standard-article(title: [Backup WordPress Database And Filesystem Data On Linux With Scripts], author: [Nic Raboy], source-name: [Nic Raboy (polyglot developer)], images: (), paragraphs: ([If you're like me, you run a WordPress blog and are terrified of the thought of something going wrong. With core updates, theme updates, plugin updates, and server component updates, there is a lot of room for error. This is where a WordPress backup could help ease your mind.], [WordPress recommends taking a backup of your blog before any of these are done and there are even some popular plugins that will do this for you. For example, you could use the popular UpdraftPlus or similar,

and some of the longer term goals of what it hopes to accomplish.], [The post Announcing The Polyglot Developer Courses Portal appeared first on The Polyglot Developer .],), insert-map: (:), word-count: 120, edited-for-length: false, debug-mode: false,)

standard-article(title: [Angular Route Guards For Authorization In A Web And Mobile Application], author: [Corbin Crutchley], source-name: [Nic Raboy (polyglot developer)], images: (), paragraphs: ([You're about to release your new Angular web app. It's a photo sharing site and you want to test it, so you send a link to it to your hacker sister. She's always messing with your stuff and she found out the URL to your admin page you added to your web app. Before you know it, she's flushed your database using a button on that admin page that you didn't restrict access to. Not a problem when using development data - but I'm sure your users wouldn't be any too keen on a service where they lost all of their data. Let's fix that], [The post Angular Route Guards For Authorization In A Web And Mobile Application appeared first on The Polyglot Developer .],), insert-map: (:), word-count: 125, edited-for-length: false, debug-mode: false,)

standard-article(title: [Upcoming Presentation: Phoenix Mobile Festival], author: [Nic Raboy], source-name: [Nic Raboy (polyglot developer)], images: (), paragraphs: ([In case I'm in your part of the world, I'll be giving a talk at the Phoenix Mobile Festival in Phoenix, Arizona on September 19th, 2015. I encourage you to attend or at least swing by and say hello.], [The title of my talk is Building Offline First Mobile Applications and the abstract is as follows:], [What happens when your mobile device does not have a network connection? That device will have no content, no experience and the amazing application that you spent so much effort to build just simply will not work.], [You will gain knowledge in NoSQL databases technologies and walk away with insights on how to create offline type of applications that to sync to the cloud with Couchbase Mobile.], [This is a free event, so if you're available and are interested in learning about mobile technologies, mark it on your calendar. Based on the listed agenda there are going to be a lot of fun topics.], [The post Upcoming Presentation: Phoenix Mobile Festival appeared first on The Polyglot Developer .],), insert-map: (:), word-count: 174, edited-for-length: false, debug-mode: false,)

standard-article(title: [Display Toast Notifications In A NativeScript Angular Application], author: [Nic Raboy], source-name: [Nic Raboy (polyglot developer)], images: (), paragraphs: ([Back when I had just started working with NativeScript, about a year ago, I wrote a tutorial for displaying Toast notifications in Android . While still valid, it demonstrated these notifications using vanilla NativeScript and JavaScript. Not only that, but the notifications were only for Android. Since the framework has become significantly better, I figured it would be a good idea to demonstrate these

but I believe there is room for error in those as well. While I could be wrong, I think WordPress must be in good shape for backup plugins to be successful.], [The alternative would be to create your own backup scripts that run on a cron schedule. We're going to see how to do this for WordPress instances running on a Linux machine.], [The post Backup WordPress Database And Filesystem Data On Linux With Scripts appeared first on The Polyglot Developer .],), insert-map: (:), word-count: 168, edited-for-length: false, debug-mode: false,)

standard-article(title: [Analyze Stack Overflow Data With Golang And HTTP], author: [Nic Raboy], source-name: [Nic Raboy (polyglot developer)], images: (), paragraphs: ([I was recently tasked with a project where I needed to gather data from Stack Overflow so it could be easily evaluated without having to dig around the website. Stack Exchange has many REST APIs available, some of which that don't even need tokens or authentication, so it came down to how I wanted to consume this data.], [In this tutorial, we're going to see how to consume question and comment data from the Stack Exchange API using Golang and then export it to comma separated value (CSV) for further evaluation.], [The post Analyze Stack Overflow Data With Golang And HTTP appeared first on The Polyglot Developer .],), insert-map: (:), word-count: 108, edited-for-length: false, debug-mode: false,)

standard-article(title: [LAMBDA: The ultimate Excel worksheet function], author: [Lambda the Ultimate], source-name: [Lambda the Ultimate], images: (), paragraphs: ([Post by Andy Gordon and Simon Peyton Jones on LAMBDA giving Excel users the ability to define functions.], [Ever since it was released in the 1980s, Microsoft Excel has changed how people organize, analyze, and visualize their data, providing a basis for decision-making for the millions of people who use it each day. It's also the world's most widely used programming language. Excel formulas are written by an order of magnitude more users than all the C, C++, C#, Java, and Python programmers in the world combined. Despite its success, considered as a programming language Excel has fundamental weaknesses. Over the years, two particular shortcomings have stood out: (1) the Excel formula language really only supported scalar values—numbers, strings, and Booleans—and (2) it didn't let users define new functions.], [Until now.],), insert-map: (:), word-count: 131, edited-for-length: false, debug-mode: false,)

standard-article(title: [TPDP Episode #23: Being A Freelance Developer Contractor Or Consultant], author: [Nic Raboy], source-name: [Nic Raboy (polyglot developer)], images: (), paragraphs: ([I am pleased to announce that the latest episode of The Polyglot Developer Podcast has been published to iTunes and all of the other popular podcasting networks! Being in the development game is tough and the competition for work is fierce. You can go down the route of working for a company as a full-time or part-time employee, or you can operate on your own.], [In the 23rd episode of

Toast notifications in a NativeScript Angular application.], [We're going to see how to use a plugin to display Toast notifications in a NativeScript Android and iOS application built with Angular.], [The post Display Toast Notifications In A NativeScript Angular Application appeared first on The Polyglot Developer .],), insert-map: (:), word-count: 111, edited-for-length: false, debug-mode: false,)

standard-article(title: [Load A JSON Configuration From File In A Golang Application], author: [Nic Raboy], source-name: [Nic Raboy (polyglot developer)], images: (), paragraphs: ([Have you ever built an application and decided that you didn't want to hardcode a bunch of values that might change frequently? The answer is, probably yes. When building a web application, it is common to separate configuration details into a separate file which might contain database information, hostnames, passwords, and anything else that probably shouldn't exist in the application as hard-coded values.], [We're going to see how to open a JSON configuration file using the Go programming language and load it into a custom data structure to be used throughout the application.], [The post Load A JSON Configuration From File In A Golang Application appeared first on The Polyglot Developer .],), insert-map: (:), word-count: 112, edited-for-length: false, debug-mode: false,)

standard-article(title: [Generate Cryptocurrency Private Keys And Public Addresses With Golang], author: [Nic Raboy], source-name: [Nic Raboy (polyglot developer)], images: (), paragraphs: ([Over the past month or so I've demonstrated how to generate address information for a variety of cryptocurrency Altcoins using technologies like Node.js, Vue.js, and Angular. The thing about my previous tutorials are that they all used the JavaScript stack in some sense. What if we wanted to adventure into other technologies like Golang?], [The process for generating key and address information for Bitcoin and popular Altcoins is pretty much the same. In reality, the difference is defined by the private key and public key prefix information, typically associated to a network.], [We're going to see how to generate and import private keys for a variety of cryptocurrency coins as well as their addresses using the Go programming language.], [The post Generate Cryptocurrency Private Keys And Public Addresses With Golang appeared first on The Polyglot Developer .],), insert-map: (:), word-count: 137, edited-for-length: false, debug-mode: false,)

standard-article(title: [Send Emails In Ionic Framework Via The Mailgun API], author: [Nic Raboy], source-name: [Nic Raboy (polyglot developer)], images: (), paragraphs: ([A few years ago I wrote a tutorial for sending emails in an Ionic Framework Android and iOS application . The tutorial works great, but it requires that a mail application be used to finalize the send. In other words, you can set default values for email fields, but you still need to select Gmail or similar to actually send the email.], [A popular question I

the show titled, Being a Freelance Developer Contractor or Consultant , I'm joined by Nolan Erck , professional freelance consultant and owner of South of Shasta Consulting in Sacramento, CA.], [The topic of this episode is working for yourself as a freelance developer consultant or contractor. In this episode Nolan and I explore everything from finding work, negotiating your rate, and ultimately accomplishing a task and building your reputation as a freelance developer.], [The post TPDP Episode #23: Being A Freelance Developer Contractor Or Consultant appeared first on The Polyglot Developer .],), insert-map: (:), word-count: 162, edited-for-length: false, debug-mode: false,)

standard-article(title: [My Activity Report For 2020], author: [Nic Raboy], source-name: [Nic Raboy (polyglot developer)], images: (), paragraphs: ([It's that time of year again. It is time to reflect on the things that happened on The Polyglot Developer , what went well, what didn't go so well, and what the upcoming year looks like.], [If this is your first time stumbling on an annual report article for The Polyglot Developer, you won't find any tutorial content. This is mostly a numbers-based reflection, but it could be useful if you're interested in building your own blog, video channel, or overall brand.], [In this particular activity report, we'll follow the same formula as the 2019 report , which includes analyzing the written content on the blog, video tutorials on YouTube, the podcast and Twitch channel, and any services and operating costs required to keep the channels alive.], [The post My Activity Report For 2020 appeared first on The Polyglot Developer .],), insert-map: (:), word-count: 141, edited-for-length: false, debug-mode: false,)

standard-article(title: [Execute HTTP Requests in JavaScript Applications], author: [Nic Raboy], source-name: [Nic Raboy (polyglot developer)], images: (), paragraphs: ([When it comes to modern application development, whether that be web, mobile, or other, there is almost always a need interact with remote web services, generally through HTTP. When working with frameworks such as Angular, Vue, and React, there is baked in functionality for making requests, but what about if you're using vanilla JavaScript or you'd prefer not to use those built in functionalities?], [In this tutorial we're going to explore a few options towards making HTTP requests in JavaScript. Particularly we're going to focus on the classic XHR request, using a modern JavaScript Fetch, as well as using a third-party package called Axios.], [The post Execute HTTP Requests in JavaScript Applications appeared first on The Polyglot Developer .],), insert-map: (:), word-count: 119, edited-for-length: false, debug-mode: false,)

standard-article(title: [Interact with a GraphQL API from a .NET Core Application], author: [Nic Raboy], source-name: [Nic Raboy (polyglot developer)], images: (), paragraphs: ([When it comes to C#, it's not too difficult or time-consuming to create or consume RESTful APIs. In fact, I recently published a tutorial that demonstrated how to create a RESTful API with .NET Core using MongoDB as the NoSQL database.], [RESTful APIs are great, but what happens when you want to work with GraphQL and query your

receive is how to send emails without launching an email application. There are a few solutions to this. You could either set up your own server that sends emails and access it via an API, or you can use a service like the Mailgun .], [We're going to see how to send emails using the Mailgun API service.], [The post Send Emails In Ionic Framework Via The Mailgun API appeared first on The Polyglot Developer .],), insert-map: (:), word-count: 142, edited-for-length: false, debug-mode: false,)

standard-article(title: [Including Attractive Charts In Your Angular Web Application], author: [Nic Raboy], source-name: [Nic Raboy (polyglot developer)], images: (), paragraphs: ([It's probably obvious, but most modern applications crave data and in many of those scenarios, being able to visualize the data is a necessity. This is where charting and graphs become valuable within frontend applications. Having attractive charts in your application can make a world of difference and with a convenient library like Chart.js , it isn't complicated.], [If you've been keeping up, you'll remember I wrote a tutorial titled, Use Chart.js to Display Attractive Charts in a Vue.js Web Application which focused on Vue.js. This time around we're going to see how to include charts in an Angular web application.], [The post Including Attractive Charts In Your Angular Web Application appeared first on The Polyglot Developer .],), insert-map: (:), word-count: 118, edited-for-length: false, debug-mode: false,)

{

Auto Attaching USB Storage To A Raspberry Pi Running Linux

Nic Raboy · Nic Raboy (polyglot developer)

I've written a few articles about how I'm using my many Raspberry Pi units. Recently I wrote about using a Raspberry Pi as an automatic network backup server , but I didn't talk about expanding the storage beyond the micro or standard sized SD card. What if you want to utilize a much larger USB hard drive or thumb drive? By default Linux won't mount the drive, and heck, it may not even be in the correct format.

We're going to see how to format a USB drive, mount it on a Raspberry Pi, and then auto mount it every time the Raspberry Pi reboots.

The post Auto Attaching USB Storage To A Raspberry Pi Running Linux appeared first on The Polyglot Developer .

}

{

ANALYSIS

APIs rather than just consume them?], [In this tutorial, we'll see how to use . NET Core to interact with a GraphQL API. To make things easier, we'll generate our API with MongoDB App Services.], [The post Interact with a GraphQL API from a . NET Core Application appeared first on MongoDB .],), insert-map: (:), word-count: 109, edited-for-length: false, debug-mode: false,)

standard-article(title: [The Polyglot Developer is Live on Twitch], author: [Nic Raboy], source-name: [Nic Raboy (polyglot developer)], images: (), paragraphs: ([I am pleased to announce that The Polyglot Developer is now a part of Twitch when it comes to streaming developer content live!], [So what can you expect when it comes to the Twitch channel?], [Think webinar, but without all the marketing nonsense that typically comes with them. Twitch streams are live, not at all formal, and an opportunity for the community to engage with the presenter as things happen on the stream. In other words, instead of leaving a comment on a blog post or YouTube video, you can ask questions and provide feedback as things happen, not only to the presenter, but to other members within The Polyglot Developer community.], [So how do you get involved and what does this mean for the other education channels?], [The post The Polyglot Developer is Live on Twitch appeared first on The Polyglot Developer .],), insert-map: (:), word-count: 144, edited-for-length: false, debug-mode: false,)

Nic Raboy, Nic Raboy (polyglot developer): Previously we've explored how to parse XML data using Node.js as well as PHP . Continuing on the trend of parsing data using various programming languages, this time we're going to take a look at parsing XML data using the dom4j library with Java.

Now dom4j, is not the only way to parse XML data in Java. There are many other ways including using the SAX parser. Everyone will have their own opinions on which of the many to use.

The post Parse An XML Response With Java And Dom4J appeared first on The Polyglot Developer .

Nic Raboy, Nic Raboy (polyglot developer): You might not know this, but I run another blog on a completely different subject material than The Polyglot Developer. This other blog is called Own the Web and it focuses on brand building, searching engine optimization, and boosting your online revenue through the internet. Unlike The Polyglot Developer, Own the Web was actually created using the Hugo Static Website Engine.

I want to share information about Hugo , some of the advantages and disadvantages, and my experience using it.

The post Use Hugo To Create Awesome Static Websites And Blogs appeared first on The Polyglot Developer .

Nic Raboy, Nic Raboy (polyglot developer): You'll notice previously I did a post regarding the Fibonacci number , a popular interview question for programming jobs. To keep up with this trend of interview questions, we're going to look into the different ways of finding duplicates in an array.

Finding array duplicates is a good question because it tests your knowledge of algorithm design and your understanding of various time complexities.

The post Calculate If Duplicates Exist In An Array Using JavaScript appeared first on The Polyglot Developer .

JEFF GANGEMI, GROWTH MARKETING PRACTICE LEAD @ TOPTAL, Tptal Engineering: As AI transforms how buyers discover and evaluate solutions, traditional SEO playbooks are being rewritten. In this episode of the Executive Guidance podcast, Kevin White, Head of Marketing at Scrunch, joins Jeff Gangemi to explore how LLMs are reshaping intent, discovery, and conversion, and what marketers should do next.

Nic Raboy, Nic Raboy (polyglot developer): When I build web applications, my least favorite part is always in the realm of file uploads. They are often complicated to do and take a lot of time. Pretty much every web application lately, regardless of what it is, requires file uploads, even if it is just to upload a profile picture.

Since I'm all about Angular lately, I figured it would be great to show how to upload images (or any file) to a back-end. In this particular example I'm using Node.js.

The post Upload Files To Node.js Using Angular appeared first on The Polyglot Developer .

Andrew Houts, Wealthfront Engineering: Ever been here before? Stuck with a job that needs to be continually revisited because its performance gets worse with every passing day, and each attempt at improving said performance yields diminishing returns? This is the situation we found ourselves in with the portfolio balance calculation system—the code responsible for aggregating data from multiple sources... Read more

Nic Raboy, Nic Raboy (polyglot developer): It has been exactly seven days since Turbo Prop for Android, iOS, Windows Phone, BlackBerry, and Amazon Kindle Fire went live. This is a status update in regards to downloads as well as ad earnings for my first Unity3D game.

The post Seven Day Turbo Prop Stats appeared first on The Polyglot Developer .

Lazarus Lazaridis (iridakos), Lazarus Lazaridis (iridakos): There are two cat beds, many chairs, a sofa but no. She will sit there.

That's how I have two step authentication for my Ubuntu.

CHRIS DANIEL, GM, CONSUMER PRODUCTS & SERVICES @ TOPTAL, Tptal Engineering: The biggest obstacle consumer packaged goods (CPG) and retail companies face isn't technology or capital. It's finding talent that can bridge the gap between business strategy and technical execution in real time.

Lambda the Ultimate, Lambda the Ultimate: LtU has experienced a long period of downtime recently. Its software infrastructure was outdated enough that it became difficult to maintain when problems arose. It has now been migrated to a brand new environment. It should be much more stable from now on.

Nic Raboy, Nic Raboy (polyglot developer): So you've just made an awesome app and you have no idea how to get the word out. This is a problem most app developers have, so you're not alone.

I've released many mobile apps to the app store, most of them Android, but still a diverse collection. It has been tough work, but I've established myself by getting many downloads, maintaining a high average rating, and getting a few hundred dollars monthly in app revenue.

The post So You Made An App, Now What? appeared first on The Polyglot Developer .

MICHAEL FIGUEROA , , Tptal Engineering: Tune into this episode of the Executive Guidance podcast for a deep dive into cutting-edge cybersecurity strategies, securing advertising platforms against digital threats, and the challenge of recruiting top cybertalent.

}

The Digital Mirror

Vol. 1, No. 085 · 2026-03-30

Curated and composed automatically.